

category: avalonia

tags: [mvvm, idisposable, cleanup, cash-acceptor, qr-payment, visa-payment, cancellation-token, back-button]

severity: critical

created: 2026-07-22

updated: 2026-07-22

project-origin: iPGSv4 (IPGS.Kiosk.Avalonia — PaymentInfoViewModel + Car payment sub-ViewModels)

Nút "Back" dùng chung ở header cast `ActivePaymentContent is IDisposable` nhưng KHÔNG sub-ViewModel nào implement IDisposable — cleanup (unsubscribe device event / hủy polling) là dead code

Tình huống gặp phải

User báo: đang thanh toán ô tô (đã đưa tiền mặt/đang chờ QR/Visa xử lý), bấm nút "Back" ở màn chọn hình thức thanh toán (`PaymentInfoView`) để quay lại — số tiền đã thanh toán (`Paid`) không được cập nhật đúng khi quay lại màn chọn phương thức.

Triệu chứng / Lỗi

Màn hình "PLEASE CHOOSE YOUR PAYMENT METHOD" sau khi bấm Back giữa lúc đang thanh toán vẫn hiển thị `Paid: 0đ / You have to pay: [tổng số tiền]` dù tờ tiền/giao dịch đã được thiết bị xử lý một phần.

Nguyên nhân gốc rễ (Root Cause)

`PaymentInfoViewModel.GoBack()` (nút Back DÙNG CHUNG ở header, hiển thị cho MỌI sub-flow Cash/QR/Visa/Voucher qua `IsBackVisible`) viết:

```
if (ActivePaymentContent is IDisposable disposable)
{
    disposable.Dispose();
}
ActivePaymentContent = null;
```

Nhưng không sub-ViewModel nào (`CarCashViewModel`, `CarQrViewModel`, `CarVisaViewModel`, `CarVoucherViewModel`) implement `IDisposable` — class khai báo chỉ : `ObservableObject`. Nhánh `is IDisposable` luôn `false`, `Dispose()` KHÔNG BAO GIỜ được gọi → đây là dead code từ khi viết, không phải regression mới.

Hệ quả cụ thể theo từng sub-flow:

- **CarCashViewModel:** `CleanUp()` (unsubscribe `CashDevice.BillReceived/DeviceError`, gọi `Disconnect()`) không chạy → driver CBA9 vẫn "connected" với handler mờ côi. Nếu người dùng chọn lại Cash sau đó, `Connect()+PollingStart()` reset object `CashResult` dùng chung bên trong driver (xem lesson liên quan `[[avalonia-cash-acceptor-reused-cashresult-stale-flag]]`) → tờ tiền đang escrow/chờ xác nhận STACKED của phiên cũ mất trạng thái, không bao giờ được cộng vào `Paid` dù máy đã nuốt tiền vật lý.
- **CarQrViewModel/CarVisaViewModel:** mỗi VM có `CancellationTokenSource _cts` cho vòng lặp `PollPaymentResultAsync` chạy nền — VM có RIÊNG 1 lệnh `GoBack()` nội bộ (link "Đổi ngân hàng"/"Change payment method") tự `_cts.Cancel()` đúng, nhưng đó là con đường back KHÁC với nút Back header dùng chung. Bấm nút Back header thay vì link riêng → `_cts` không bị hủy → vòng poll vẫn chạy ngầm sau khi rời màn, có thể gọi `_callback.OnPaymentCompleted` trễ trên callback đã ở trạng thái khác, hoặc bỏ sót ghi log hủy giao dịch.

Giải pháp

```
// CarCashViewModel / CarQrViewModel / CarVisaViewModel
public sealed partial class CarCashViewModel : ObservableObject, IDisposable
{
    ...
    private bool _cleanedUp;
    private void CleanUp()
    {
        if (_cleanedUp) return;
        _cleanedUp = true;
        _blinkTimer?.Stop();
        KioskServices.CashDevice.BillReceived -= OnBillReceived;
        KioskServices.CashDevice.DeviceError -= OnDeviceError;
        KioskServices.CashDevice.Disconnect();
    }
    public void Dispose() => CleanUp(); // giờ nhánh is IDisposable ở parent chạy
    thật
}
```

Với Qr/Visa: `Dispose()` chỉ hủy `_cts` + dừng dot-animation timer (guard bằng `_goBackCalled` sẵn có) — KHÔNG gọi lại `_callback.OnPaymentCancelled` vì parent đã tự set `ActivePaymentContent = null` ngay sau khi `Dispose()` chạy xong.

1. Xác định tất cả nơi 1 ViewModel cha dùng pattern `object? ActivePaymentContent + cast is IDisposable` để cleanup polymorphic cho các sub-ViewModel.
2. Liệt kê toàn bộ class có thể được gán vào slot đó — kiểm tra từng class có thực sự implement interface mà parent cast tới hay không (đừng tin comment/tên biến).
3. Build không báo lỗi gì cả vì `is` pattern match false chỉ skip nhánh, không phải lỗi biên dịch — bug này KHÔNG lộ ra qua build hay qua test đường happy-path (chọn phương thức → hoàn tất bình thường không bao giờ chạm nhánh Back giữa chừng).

Áp dụng lại (How to reuse)

- Khi review code có pattern `if (x is IDisposable d) d.Dispose();` (hoặc tương tự cast-to-interface-rồi-gọi) — LUÔN tự hỏi: "class thực tế được gán vào biến này có implement interface đó không?" Grep `class.*:.*ObservableObject` (hoặc base class tương ứng) để xác nhận, đừng giả định.

- Bất kỳ sub-ViewModel nào có tài nguyên cần dọn (event subscription, `CancellationTokenSource`, `DispatcherTimer`, kết nối thiết bị) và được host trong 1 slot `object?/ContentControl.Content` dùng chung nhiều loại — PHẢI implement `IDisposable` tường minh, không dựa vào cleanup nội bộ của riêng command back/cancel của chính nó (vì có thể có ĐƯỜNG THOÁT KHÁC — nút back dùng chung, đóng cửa sổ, timeout countdown — không đi qua command đó).
- Khi 1 sub-ViewModel có sẵn logic cleanup trong chính command `GoBack()`/`Cancel()` nội bộ của nó — refactor tách phần "dọn tài nguyên" (cancel token, stop timer, unsubscribe event) ra khỏi phần "raise callback / đổi state điều hướng", để `Dispose()` gọi phần dọn tài nguyên mà không raise callback trùng lặp.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Bug loại "polymorphic cleanup qua cast interface nhưng object thật không implement" KHÔNG gây lỗi build, KHÔNG gây exception runtime — chỉ âm thầm skip, cực dễ lọt qua code review nếu không grep xác nhận từng implementer.
- ⚠ Nếu 1 sub-ViewModel có 2 con đường "back" khác nhau (nút riêng bên trong view con + nút dùng chung ở header/parent), PHẢI đảm bảo CẢ HAI con đường dẫn tới cùng logic cleanup — không chỉ path được test nhiều nhất (thường là nút riêng, vì đó là happy-path chính được QA test) mới hoạt động đúng.
- ⚠ Khi thêm guard chống double-cleanup (`_cleanedUp/_goBackCalled`), nhớ đặt guard đó dùng chung giữa CHÍNH command back nội bộ VÀ `Dispose()` mới thêm — nếu không sẽ chạy cleanup 2 lần (ví dụ unsubscribe event 2 lần — vô hại với `--` nhưng gọi `Disconnect()`/`_cts.Cancel()` 2 lần nên vẫn nên guard cho sạch).

Tham chiếu

- File liên quan: `IPGS.Kiosk.Avalonia/ViewModels/Payment/PaymentInfoViewModel.cs` (`GoBack` — nơi có cast `is IDisposable`)
- Đã fix: `CarCashViewModel.cs`, `CarQrViewModel.cs`, `CarVisaViewModel.cs` (thêm `IDisposable`)
- Chưa cần fix: `CarVoucherViewModel.cs` (không giữ tài nguyên cần dọn — `GoBack()` nội bộ đã đủ)
- Lesson liên quan: `[[avalonia-cash-acceptor-reused-cashresult-stale-flag]]` (driver CBA9 reset `CashResult` khi `Connect()/PollingStart()` lại)
- Project liên quan: iPGSv4 (T-F12 migrate WinForms → Avalonia)